

An Analysis of Transformer architecture deep learning

Transformer architecture deep learning

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Transformer architecture deep learning -- Part 1

Each integer token identifier is converted into an embedding vector via a lookup table. Equivalently stated, it multiplies a one-hot representation of the token identifier by an embedding matrix M . For example, if the input token's identifier is 3, then the one-hot representation is $[0, 0, 0, 1, 0, 0, \dots]$, and its embedding vector is $\text{Embed}(3) = [0, 0, 0, 1, 0, 0, \dots] M$. The token embedding vectors are added to their respective positional encoding vectors (see below), producing the sequence of input vectors. The dimension of an embedding vector is called hidden size or embedding size and written as d_{emb} . This size is written as d_{model} in the original transformer paper.

Transformer architecture deep learning -- Part 2

The un-embedding layer is a linear-softmax layer: $\text{UnEmbed}(x) = \text{softmax}(xW + b)$. The matrix has shape $(d_{\text{emb}}, |V|)$. Some architectures use the transpose of the embedding matrix M as the un-embedding matrix W in order to avoid needing double the amount of embedding-related parameters and to avoid divergence during training. This practice is called weight tying.

Transformer architecture deep learning -- Part 3

Seq2seq models with attention (including self-attention) still suffered from the same issue with recurrent networks, which is that they are hard to parallelize, which prevented them from being accelerated on GPUs. In 2016, decomposable attention applied a self-attention mechanism to feedforward networks, which are easy to parallelize, and achieved SOTA result in textual entailment with an order of magnitude fewer parameters than LSTMs. One of its authors, Jakob Uszkoreit, suspected that attention without recurrence would be sufficient for language translation, thus the title "attention is all you need". That hypothesis was against conventional wisdom at the time, and even his father Hans Uszkoreit, a well-known computational linguist, was skeptical. In the same year, self-attention (called intra-attention or intra-sentence attention) was proposed for

LSTMs. On 2017-06-12, the original (100M-parameter) encoder-decoder transformer model was published in the "Attention is all you need" paper. At the time, the focus of the research was on improving seq2seq for machine translation, by removing its recurrence to process all tokens in parallel, but preserving its dot-product attention mechanism to keep its text processing performance. This led to the introduction of a multi-head attention model that was easier to parallelize due to the use of independent heads and the lack of recurrence. Its parallelizability was an important factor to its widespread use in large neural networks.

Transformer architecture deep learning -- Part 4

Alternative activation functions The original transformer uses ReLU activation function. Other activation functions were developed. The Llama series and PaLM used SwiGLU; both GPT-1 and BERT used GELU. Alternative activation functions are often used in combination with Gated Linear Units in the feedforward module.

Transformer architecture deep learning -- Part 5

Random Feature Attention Random Feature Attention (2021) uses Fourier random features: $\phi(x) = \frac{1}{\sqrt{D}} [\cos(w_1, x), \sin(w_1, x), \dots, \cos(w_D, x), \sin(w_D, x)]^T$ where $w_1, \dots, w_D$ are independent samples from the normal distribution $N(0, \sigma^2 I)$. This choice of parameters satisfy $E[\phi(x), \phi(y)] = e^{-\frac{\|x-y\|^2}{2\sigma^2}}$, or $e^{\frac{\langle x, y \rangle}{\sigma^2}} = E[\phi(x), \phi(y)] \approx e^{\frac{\langle x, y \rangle}{\sigma^2}}$. Consequently, the one-headed attention, with one query, can be written as $\text{Attention}(q, K, V) = \text{softmax}\left(\frac{qK^T}{\sqrt{d_k}}\right)V \approx \frac{1}{\sqrt{d_k}} \sum_i e^{\frac{\langle q, k_i \rangle}{\sigma^2}} v_i$ where $\sigma =$

An Analysis of Transformer architecture deep learning

Transformer architecture deep learning

$\sigma = d_K^{1/4}$. Similarly for multiple queries, and for multihead attention. This approximation can be computed in linear time, as we can compute the matrix $\phi(k_i) v_i^T$ first, then multiply it with the query. In essence, we have managed to obtain a more precise version of Attention $(Q, K, V) = \text{softmax}(QK^T / d_k) V \approx Q (K^T V / d_k)$ $\{\text{Attention}\}(Q,K,V)=\{\text{softmax}\}\left(\frac{QK^T}{d_k}\right)V\approx Q(K^TV/\sqrt{d_k})$ Performer (2022) uses the same Random Feature Attention, but $w_1, \dots, w_D$ are first independently sampled from the normal distribution $N(0, \sigma^2 I)$, then they are Gram–Schmidt processed.

Transformer architecture deep learning -- Part 6

```
/* first sublayer */ z_d_copy ← copy(z_d) for each t in
1:length(z_d) do z_d[t] ← layer.layer_norm(z_d[t]) z_d ←
layer.masked_multihead_attention(z_d, z_d, z_d) for each t in
1:length(z_d) do z_d[t] ← z_d[t] + z_d_copy[t]
```